SELF-HEALING INFRASTRUCTURE

Project Report | Automated Monitoring, Alerting and Recovery

1. Introduction

Service downtime can interrupt user access and affect the reliability of applications. This project demonstrates a basic self-healing workflow for an NGINX web server using monitoring, alerting, and automation tools. When the HTTP health check fails, the system raises an alert and attempts to restore the service without requiring a person to manually restart the container.

2. Abstract

The project integrates Prometheus, Blackbox Exporter, Alertmanager, a Python Flask webhook, Docker, and Ansible. Blackbox Exporter checks the NGINX endpoint. Prometheus evaluates the **NginxDown** alert rule, and Alertmanager forwards firing alerts to the webhook. Flask invokes an Ansible playbook, which restarts the existing NGINX Docker container and checks that the service returns HTTP 200. The workflow was tested by stopping the NGINX container and observing its automated restart and successful health check.

3. Tools Used

Tool	Purpose
Docker & Docker Compose	Run and manage NGINX and the monitoring services in containers.
Blackbox Exporter	Probe the NGINX HTTP endpoint and report whether it is reachable.
Prometheus	Collect probe metrics and evaluate alert rules.
Alertmanager	Route firing alerts to the Flask webhook.
Python & Flask	Receive alert webhooks and trigger recovery automation.
Ansible	Restart the existing NGINX container and verify recovery.

4. Steps Involved in Building the Project

- **1. Set up NGINX:** Run an NGINX container and expose it on host port 8081.
- **2. Configure health monitoring:** Set up Blackbox Exporter to probe NGINX and configure Prometheus to scrape the probe metrics.
- **3. Define an alert:** Create the **NginxDown** rule so Prometheus fires an alert when the HTTP probe fails for the configured duration.
- **4. Configure alert routing:** Set Alertmanager to send firing alerts to the Flask webhook on the host machine.
- **5. Implement recovery:** Have Flask invoke an Ansible playbook when it receives the NginxDown alert.
- **6. Verify recovery:** Use Ansible to restart the existing container and retry the HTTP check until it returns status 200.
- **7. Test the workflow:** Stop NGINX, confirm the alert reaches the webhook, and verify that the container comes back up.

5. Conclusion

The project successfully demonstrates a monitoring-to-recovery pipeline: a failed health check is detected, an alert is routed to a webhook, and Ansible restarts the existing NGINX container and verifies that it responds. It provided practical experience with infrastructure monitoring, alert routing, Docker, Python webhook integration, and automation. The current implementation is a local learning demonstration; it restarts an existing container and does not recreate a deleted container. Production use would require stronger security, persistent recovery state, and additional resilience controls.

Repository: https://github.com/AyushShahir/self-healing-infrastructure